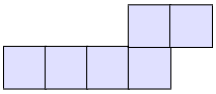
SNAKY WITHIN NINETEEN MAKER MOVES

Abstract. We give a finite, independently checked certificate that Maker can force Snaky within nineteen of Maker's own moves on the initially empty infinite square board. The same certificate proves the bound on a 17×17 board. It combines Nándor Sieben's twenty-move strategy with local tactics from Trevor Morris's 15×15 certificate. The construction retains all acquired Maker stones and tracks a larger region known to contain no Breaker stone, allowing previously discarded safe cells to be used again. The resulting 3,468-card certificate uses the existing declarative rules. Its soundness follows by induction on card height, with a universal response class for moves outside the support. The complete certificate and the infinite-board soundness theorem have also been checked by the Lean kernel. We make no optimality claim.

1. Game and theorem

The board is either $\mathbb{Z}^2$ or $Q = \{0, \dots, 16\}^2$. Maker moves first. On each turn the player claims exactly one previously unclaimed cell. Breaker only blocks; Breaker completing a shape does not end the game. Maker wins upon owning a translated, rotated, or reflected copy of

$$F = \{(0,0), (1,0), (2,0), (3,0), (3,1), (4,1)\}. \tag{1}$$



Other Maker stones are allowed: the six-cell condition specifies a winning subset. A move bound counts actual Maker claims, including the winning claim, and does not count alternating plies.

Snaky arises in Harary's polyomino achievement games. Sieben proved that it wins in 41 dimensions [2], and Halupczok and Schlage-Puchta proved that it wins in three dimensions and has planar handicap number at most one [1]. OpenAI proved an ordinary planar win within 21 Maker moves [4], and Sieben improved this to 20 [3].

Theorem 1.1. *From the empty board, Maker has a strategy that completes a copy of F by Maker's nineteenth move against every legal Breaker strategy. This holds both on $\mathbb{Z}^2$ and on Q.*

The data establishing the theorem are `certificate/snaky19.cards.txt`, with SHA-256

`baf8774e1fc6eea60b1756437e8b3caae0ede5e001b10c0c3787aab51840782e`

The file contains 3,468 cards, all reachable from root 3467. The root has height 19, no required Maker stones, first move $(8, 8)$, and a 249-cell support contained in Q. Its longest dependency route also has height 19.

2. Declarative certificate and soundness

A card is a tuple $C = (A, S, p, h)$: finite integer-cell sets A and S, a pivot $p \in S$, and a positive integer height h. Its intended meaning is that Maker can win within h further claims whenever Maker owns A and Breaker owns no cell of S. Extra Maker stones are permitted. Cards can be placed by any map $g(x) = Dx + t$, where D is one of the eight symmetries of the square and $t \in \mathbb{Z}^2$.

A card is terminal if $A \cup \{p\}$ contains a copy of F. Otherwise it must provide the following hints. Each hint selects another card (A', S', p', h') and a map g, with

$$h' < h, \qquad g(A') \subseteq A \cup \{p\}. \tag{2}$$

Date: 15 October 2026.

For every $w \in S \setminus (A \cup \{p\})$, an inside-response hint must additionally satisfy $g(S') \subseteq S \setminus \{w\}$. An outside-response hint must satisfy $g(S') \subseteq S$. The latter is one universally valid rule, not an assumption that Breaker can pass. It applies to every legal claim outside S, including holes in its bounding rectangle and points arbitrarily far away.

The finite checker verifies these inclusions, all required response classes, the coordinate maps, terminal geometry, and strict height decreases. The construction algorithm is not part of the certificate's logical premises.

Lemma 2.1 (Soundness)**.** *Let every card satisfy the preceding rules. At a Maker turn on $\mathbb{Z}^2$, let M, W be the finite, disjoint ownership sets. If an instance of a height-h card satisfies $g(A) \subseteq M$ and $g(S) \cap W = \varnothing$, Maker can complete Snaky within at most h additional claims.*

Proof. Induct strongly on h. A position already containing Snaky is already won. Otherwise, if $g(p)$ is unowned, Maker claims it; the claim is legal because $p \in S$ and the support is Breaker-free. If $g(p)$ is already Maker's, Maker instead claims any fresh cell. Such a cell exists because the histories are finite and the board is infinite. This is an actual claim and is charged to the height budget. In either case the resulting Maker set contains $g(A \cup \{p\})$.

If the terminal condition holds, Maker now owns the required six-cell copy. Otherwise consider an arbitrary legal Breaker claim w. If $w \in g(S)$, its unique preimage is outside $A \cup \{p\}$, since Breaker cannot claim Maker's stones. The corresponding checked hint gives a child whose required stones are owned and whose support avoids both the old Breaker stones and w. If $w \notin g(S)$, the outside hint supplies the same conclusion. Composition of square symmetries and translations preserves the allowed maps and winning shapes. The child has height at most $h - 1$; induction therefore bounds the total by one Maker claim plus $h - 1$. These two reply cases exhaust $\mathbb{Z}^2$. □

For the finite board, the same proof uses the capacity invariant

$$|M| + |W| + 2h \leq |Q|. \tag{3}$$

It supplies a free cell for an already-owned pivot's replacement claim and for a nonterminal Breaker reply. After one claim by each player and a strict height decrease, the invariant persists. Root support containment keeps all subsequent supports on the board; child requirements are contained in already owned stones. At the empty root, $2 \cdot 19 \leq 289$. Applying Lemma 2.1 directly on the plane, and its finite version on Q, proves both assertions of Theorem 1.1. The infinite-board conclusion does not rely on treating outside-board Breaker moves as legal passes in a finite game.

3. Construction by retaining history and safe space

The starting twenty-move certificate has 1,738 cards. Reconstructing its dependency graph gives 15,696 maximum-height labelled routes through 59 critical cards. Full-history replay distinguishes 9,792 routes that repeat a planned Maker stone, 2,592 with an already-occupied Breaker reply, 864 with an earlier global win, and 2,448 genuine twenty-move routes. Thus simply correcting this bookkeeping does not establish nineteen moves.

Fix the initial baseline support U. During construction retain the full Maker set M and the actual Breaker set within U, giving the safe region $E = U \setminus W$. A local template may forget stones and reduce its support, but these known facts about the actual position remain available.

For a remaining budget b, the constructor tries a transformed catalog card with height at most b, requirements contained in M, and support contained in E. If one fits, its certified strategy closes that node. Otherwise the constructor follows the baseline move. Replies whose existing hints already fit the smaller budget retain those hints; the other replies are processed recursively. Every claim inside U remains a separate possible update of W, even when outside the current template. Only claims outside U are merged, since every future support remains inside U.

Using only the twenty-move catalog left an unresolved branch. Adding the 8,671-card catalog underlying Morris's 15×15 result closed the whole root. That catalog's root has height 25; its usefulness here lies in its local tactics, not in a better standalone move bound.

The successful construction uses 180 catalog substitutions. All 180 reclaim safe cells outside the current baseline support; 68 also use Maker stones forgotten by the baseline requirements. Seventeen distinct 15×15 tactics account for 164 substitutions, and three twenty-move tactics account for the remaining 16. No zero-cost repeated-pivot shortcut is used in this run.

Three early substitutions illustrate how the surviving long branches are removed. IDs below are zero-based IDs in the public 15×15 catalog. Coordinates are in the final root frame, immediately before Maker's next claim.

(1) With $M = \{(8,8),(8,9)\}$ and $W = \{(7,8),(9,9)\}$, replace baseline card 1702 of height 18 by source card 28 of height 17 under

$$g(x,y) = (y+2, x+3).$$

Its next move is $(8,10)$. Two Maker claims have already occurred, so this continuation has total bound 19.

(2) With $M = \{(8,8),(8,9),(11,8)\}$ and $W = \{(7,8),(8,10),(12,8)\}$, replace baseline card 1036 of height 17 by source card 79 of height 15 under

$$g(x,y) = (14-x, y+3).$$

Its next move is $(11,9)$, giving total bound 18 on this branch.

(3) For the same M and $W = \{(7,8),(8,10),(13,8)\}$, use source card 145 of height 14 under

$$g(x,y) = (y+4, 12-x).$$

Its next move is $(9,8)$, giving total bound 17 on this branch.

These examples do not replace verification of the remaining replies. The constructor exports every new decision as an ordinary card, initially with requirements M and support E. It then discards support cells unused by its pivot, requirements, and child supports. Reachability pruning and reindexing leave 1,662 twenty-move-source cards, 1,653 15×15-source cards, and 153 newly composed cards. Exact source IDs and transformation witnesses are recorded in the constructor documentation.

4. Computational evidence and reproducibility

The independent Python verifier shares no constructor checking helpers. For the exact delivered file it checked 61,040 inside-response obligations, 3,456 universal outside-response obligations, and twelve terminal cards. It accepted empty root requirements, support containment, and height 19.

The verifier's tests include malformed terminal shapes, missing inside or outside replies, invalid transforms, broken ownership/support inclusions, invalid references, and incorrect height descent. Controls applied to the final file rejected both lowering the root height to 18 and deleting its reply at $(0,4)$. The first rejection concerns this certificate's declared dependencies; it does not prove that an eighteen-move strategy is impossible.

The deterministic construction used Python 3.9.6, one computational thread, no random seed, a 120-second wall limit, and a two-million-call cap. The successful run took about fifteen seconds. The certificate can be checked offline using only the Python standard library:

```
python3 topics/snaky/reproduce.py
```

Adding `-reconstruct` regenerates the certificate from the pinned source files and requires byte-for-byte identity. The final reproduction passed all 31 tests and both corruption controls, and regenerated identical bytes.

Lean 4.34.1 completed the full kernel check of the generated certificate and the direct plane soundness development in `formal/Snaky/Plane.lean`. The file `formal/Snaky/Result19.lean` proves

```
Snaky.snaky_wins_17x17_in_19
  Snaky.snaky_wins_plane_in_19 :  Snaky.PlaneBlackWinsIn 19 [] []
```

The latter theorem quantifies over every legal integer-coordinate Breaker reply directly. Both theorems report exactly `propext`, `Classical.choice`, and `Quot.sound` as their axioms. Data checks use `decide +kernel`, with no proof placeholders, custom axioms, or `native_decide`. This formal proof does not rely on the correctness of the Python constructor, verifier, or generator: their output obligations are checked by the Lean kernel. Exact theorem output and build details appear in `formal/kernel-audit.json`; the formal definitions, Lean kernel, and compiler/runtime remain part of the formal trust boundary.

5. Attribution, novelty, and scope

The twenty-move strategy is due to Nándor Sieben [3], building on OpenAI's twenty-one-move strategy [4]. Trevor Morris's repository [5] supplies the compact card interface, its soundness development, the converted twenty-move certificate, and the 15×15 tactic catalog. We reuse and credit those tactics; the contribution here is their history-aware composition into a uniform nineteen-move certificate. Pinned revisions, hashes, and the retained upstream license are documented in `reference/README.md`.

The 15 October 2026 novelty audit inspected these primary sources, the public bounds tracker [6], and searches for nineteen-move-or-better strategies. It located twenty moves as the strongest public bound and found no matching nineteen-move result. This is a bounded audit, not proof that no unindexed or unpublished result exists. We establish an upper bound and claim neither optimality nor any global Breaker lower bound.

Two further public projects concern the twenty-move bound, and we do not use either. Erebos Labs released Lean soundness code for Sieben's strategy on the infinite lattice [7]. KaiaSage published an independent draft twenty-move certificate. Its research notes report that the plan studied there is refuted at nineteen moves [8].

References

[1] Immanuel Halupczok and Jan-Christoph Schlage-Puchta, *Achieving Snaky*, Integers **7** (2007), G02, `math.colgate.edu/~integers/hg2/hg2.pdf`.

[2] Nándor Sieben, *Snaky is a 41-dimensional winner*, Integers **4** (2004), G05, `math.colgate.edu/~integers/eg5/eg5.pdf`.

[3] Nándor Sieben, *Snaky*, twenty-move proof sequence, `github.com/nandorsieben/Snaky`, revision `a0a36dd4edffdccdb7c4b14849c81e83dd6a25aa`, 8 October 2026.

[4] OpenAI, *Snaky in 21 Maker moves*, manuscript dated 25 September 2026, `github.com/openai/math`, repository revision `1581d4cbdd69a0faf34b2089614fe2a814316a6a`, 12 October 2026.

[5] Trevor Morris, *Snaky on a* 15×15 *board*, card certificates and Lean soundness developments, `github.com/gotrevor/snaky-15x15`, revision `b18ed32473a6a22ebfdda98da62d61b8b1590c1e`, 12 October 2026.

[6] Bigospeedrun, public bounds tracker, `src/lib/other-bounds.json`, `github.com/bigospeedrun/site`, revision `88d1855ba6843586579d8e031389ad94a475bb41`, inspected 15 October 2026.

[7] Erebos Labs, *Snaky bounded soundness code and generators*, `github.com/erebos-labs/snaky-bounded-lean`, revision `239dae48c0f8e227595424a9318126e01508df3a`, 9 October 2026.

[8] KaiaSage, *Snaky in 21* and draft twenty-move certificate, `github.com/KaiaSage/snaky-demo`, research branch revision `db62e51521944b0a7101062567352532efcec069`, 8 October 2026.